Politechnika Warszawska

WYDZIAŁ ELEKTRYCZNY

Instytut Elektrotechniki Teoretycznej i Systemów Informacyjno-Pomiarowych

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana

w specjalności Inżynieria Oprogramowania

Analiza metod mitygacji podatności dużych modeli językowych

Filip Sosnowski

numer albumu 319105

promotor

dr inż. Robert Szmurło

WARSZAWA 2026

Analiza metod mitygacji podatności dużych modeli językowych

Streszczenie

Słowa kluczowe:

Analysis of methods for mitigating vulnerabilities in Large Language Models

Abstract

Keywords:

Spis treści

1 Wstęp **9**
- 1.1 Motywacja . . . 9
- 1.2 Cel i zakres pracy . . . 9
- 1.3 Zawartość pracy . . . 10

2 Ataki na duże modele językowe **11**
- 2.1 Zagrożenia bezpieczeństwa . . . 11
- 2.2 Klasyfikacja ataków promptowych . . . 12
- 2.3 Metody bezpośrednich ataków promptowych . . . 13
  - 2.3.1 Ataki oparte na optymalizacji . . . 13
  - 2.3.2 Metody manualne . . . 13
  - 2.3.3 Automatyczny red teaming oparty na modelach . . . 14

3 Metody obrony przed atakami na duże modele językowe **17**
- 3.1 Klasyfikacja metod obrony . . . 17
  - 3.1.1 Metody stosowane na etapie trenowania modelu . . . 17
  - 3.1.2 Metody stosowane na etapie wdrożenia modelu . . . 18
  - 3.1.3 Metody stosowane na etapie ewaluacji modelu . . . 19
- 3.2 Inżynieria aktywacji jako metoda obrony . . . 19
  - 3.2.1 Podstawy teoretyczne inżynierii aktywacji . . . 19
  - 3.2.2 Generowanie wektora i sterowanie modelem . . . 20
  - 3.2.3 Zapobieganie atakom za pomocą inżynierii aktywacji . . . 21
  - 3.2.4 Ograniczenia metody . . . 22

4 Projekt i implementacja środowiska badawczego **23**
- 4.1 Cel i założenia eksperymentu . . . 23
- 4.2 Środowisko sprzętowe . . . 23
- 4.3 Modele językowe . . . 24
- 4.4 Zbiory danych . . . 24
- 4.5 Metody ataku . . . 25
  - 4.5.1 DAN . . . 25
  - 4.5.2 GCG . . . 25

4.5.3 PAIR . . . 26
4.6 Metody obrony . . . 26
4.6.1 Self-Reminder . . . 26
4.6.2 Llama-Guard . . . 26
4.6.3 Sterowanie aktywacjami . . . 28
4.7 Metryki oceny . . . 29

5 Analiza wyników badań **31**
5.1 Skuteczność ataków bez zastosowania metod mitygacji . . . 31
5.2 Skuteczność metod obrony . . . 33
5.3 Analiza kompromisu między bezpieczeństwem a użytecznością modelu . . . 37
5.4 Analiza opóźnień wprowadzanych przez obronę . . . 37
5.5 Dyskusja wyników i ograniczenia badania . . . 37

6 Podsumowanie **39**
6.1 Wnioski . . . 39
6.2 Możliwości dalszych badań . . . 39

Bibliografia **41**

Wykaz skrótów **45**

Spis rysunków **47**

Spis tabel **49**

Rozdział 1

Wstęp

1.1 Motywacja

Motywacją do podjęcia tego tematu jest między innymi globalny wzrost zainteresowania technologiami wykorzystującymi sztuczną inteligencję (ang. Artificial Intelligence (AI)). Według raportu przygotowanego przez Uniwersytet Stanforda, generatywna sztuczna inteligencja (ang. Generative Artificial Intelligence (GenAI)) została zaadaptowana przez 53% populacji w ciągu trzech lat, co czyni ją jedną z najszybciej wdrażanych technologii w historii [30]. Ważną częścią tych systemów stają się duże modele językowe (ang. Large Language Model (LLM)), które używane są do tworzenia zaawansowanych wyszukiwarek, chatbotów czy narzędzi wspomagających wytwarzanie oprogramowania [33]. Stosowanie modeli językowych na skalę masową sprawia, że zagwarantowanie ich bezpieczeństwa ma wyjątkową wagę. Spośród zidentyfikowanych zagrożeń dla systemów opartych na LLM, szczególne znaczenie mają ataki typu wstrzykiwanie poleceń (ang. prompt injection). Polegają na celowym manipulowaniu zachowaniem lub odpowiedzią modelu językowego w sposób niezamierzony poprzez polecenie użytkownika [25]. Istotność tego ryzyka potwierdza klasyfikacja przygotowana przez organizację Open Worldwide Application Security Project (OWASP), która w swoim zestawieniu uznaje tego typu ataki za największe zagrożenie dla bezpieczeństwa we współczesnych aplikacjach wykorzystujących modele językowe [25]. Z tego względu główną motywacją do podjęcia niniejszych badań jest chęć analizy i porównania skuteczności metod mitygacji tej krytycznej podatności w obliczu szybkiego tempa integracji systemów GenAI i potrzeby zapewnienia wysokiego poziomu bezpieczeństwa dużych modeli językowych.

1.2 Cel i zakres pracy

Głównym celem pracy jest przeprowadzenie analizy porównawczej i ocena skuteczności wybranych metod mitygacji podatności dużych modeli językowych na ataki typu prompt injection. Realizacja wymaga implementacji zróżnicowanych podejść do obrony, takich jak obrona na poziomie wejścia, wykorzystanie zewnętrznych modeli moderacyjnych czy modyfikacja aktywacji modelu przy użyciu

inżynierii aktywacji. W celu sprawdzenia bezpieczeństwa modeli należy również poddać je różnym wektorom ataku. Ważnym krokiem jest również opracowanie procedury testowej opartej na zbiorze danych zawierającym zapytania niedozwolone oraz bezpieczne. Rezultatem pracy będzie wskazanie optymalnych mechanizmów obronnych dla wybranych modeli językowych, zapewniających wysoki poziom bezpieczeństwa przy zachowaniu zadowalającej jakości ich działania.

1.3 Zawartość pracy

Praca składa się z x rozdziałów. We wstępie omówiono motywację do podjęcia tematu, określono cele do zrealizowania oraz opisano strukturę pracy. Następny rozdział zawiera ... . W rozdziale trzecim omówiono ... . W podrozdziałach czwartej części ...

Rozdział 2

Ataki na duże modele językowe

2.1 Zagrożenia bezpieczeństwa

Wraz z rozwojem dużych modeli językowych pojawiają się ich nowe, groźne podatności. Udoskonalaniu mechanizmów bezpieczeństwa towarzyszy również rozwój coraz bardziej zaawansowanych metod ataku. Stanowi to istotne wyzwanie dla twórców systemów AI, którzy dążą do rozbudowy funkcjonalności modeli jednocześnie minimalizując ryzyko ich nadużycia. Potężne możliwości generowania tekstu przez duże modele językowe niosą ze sobą również zagrożenie takie jak tworzenie szkodliwych treści, zarówno w sposób nieintencjonalny jak i w wyniku przeprowadzonego ataku [41]. Generowane w ten sposób niedozwolone odpowiedzi można uporządkować w cztery kategorie [41]. Jedną z nich może być toksyczność obejmująca wulgaryzmy, wyzwiska czy uprzedzenia kulturowe. Drugą kategorią jest nękanie, którego celem jest zastraszanie czy kierowanie gróźb. Kolejną kategorią jest mowa nienawiści. Ostatnią kategorię tworzą treści obarczone uprzedzeniami, które skierowane są do osób o określonej rasie, płci, religii czy orientacji. Rozwój bezpieczeństwa dużych modeli językowych jest kluczowy, by niwelować możliwość generowania takich treści i dostarczania ich użytkownikom.

Według organizacji OWASP najistotniejszym zagrożeniem dla dużych modeli językowych są ataki typu wstrzykiwanie poleceń (ang. prompt injection) [25]. Podatność ta występuje, gdy dane wejściowe przekazane przez użytkownika modyfikują zachowanie lub odpowiedź modelu w sposób nieprzewidziany przez twórców. Skutki udanego ataku mogą wykraczać poza samo generowanie szkodliwych treści Mogą prowadzić do zakłóceń prawidłowego działania modelu, blokowania funkcjonalności, skłonienia systemu do odbiegania od pożądanego zachowania oraz wycieku prywatnych danych przetwarzanych przez LLM [33]. Zagrożeniem wynikającym z tego typu ataków może być również obejście mechanizmów obronnych i zasad etycznych wbudowanych przez twórców modelu, przejęcie kontroli nad wywoływaniem zewnętrznych narzędzi, czy również wydobycie ukrytych informacji z kontekstu rozmowy, takich jak przykładowo instrukcja systemowa (ang. system prompt) [33].

2.2 Klasyfikacja ataków promptowych

Ataki promptowe można sklasyfikować między innymi według miejsca wstrzyknięcia złośliwej instrukcji. Wyróżnić można ataki pośrednie (ang. indirect prompt injection), w których system przyjmuje dane wejściowe ze źródeł zewnętrznych, przykładowo z plików, stron internetowych czy wiadomości e-mail zawierających treść, która może wpłynąć na zachowanie modelu w sposób nieoczekiwany bez bezprośredniego udziału użytkownika [25]. Obok ataków pośrednich, istotną podgrupą zagrożeń są również bezpośrednie ataki promptowe (ang. direct prompting attacks). Występują w scenariuszu, w którym atakujący jest głównym użytkownikiem systemu i wchodzi w interakcję z modelem poprzez interfejs zapytań, by skłonić model do zignorowania wbudowanych reguł bezpieczeństwa [33]. Podklasą ataków bezpośrednich są ataki typu jailbreak (tłum. dosł. ucieczka z więzienia). Pojęcia prompt injection oraz jailbreak w literaturze bywają stosowane zamiennie, choć różnią się zakresem [25]. Wstrzykiwanie polecenia polega na manipulacji danymi wejściowymi w celu zmiany logiki i zachowania modelu, ale nie musi to obejmować ominięcia środków obronnych [25]. Z kolei ataki typu jailbreak koncentrują się na umożliwieniu nieuprawnionego użycia poprzez obejście wbudowanych w modelu mechanizmów odmowy, co może ułatwić realizację kolejnych, bardziej złożonych ataków [11]. Ataki promptowe można również kategoryzować na podstawie dostępu do modelu. Wyróżnić w ten sposób można ataki białoskrzynkowe, w ramach których atakujący ma pełny dostęp do parametrów modelu, danych treningowych, wag, czy architektury [10]. Atakom tym podlegają przede wszystkim modele udostępniane do pobrania z ogólnodostępnych źródeł, przykładowo z platformy Hugging Face. Natomiast w przypadku ataków czarnoskrzynkowych nie ma możliwości dostępu do parametrów modelu, więc atak przeprowadzany jest z minimalną wiedzą o systemie lub jej całkowitym brakiem [33]. Tego typu atak oparty jest na interakcji z modelem i analizie zwracanych przez niego odpowiedzi, co stanowi najbardziej prawdopodobny scenariusz ataku w praktycznych zastosowaniach modeli językowych. Innym kryterium klasyfikacji ataków promptowych może być przebieg interakcji z modelem. Wyodrębnić można konwersacje jednoturowe (ang. single-turn attacks), których celem jest stworzenie pojedynczego, samowystarczalnego polecenia wejściowego do modelu, które natychmiastowo przełamuje zabezpieczenia modelu. Wyróżnić można też interakcje wieloturowe (ang. multi-turn attacks), w których atakujący wykorzystuje stan konwersacji i okno kontekstowe modelu, by przy użyciu sekwencji zapytań, krok po kroku skłonić model do naruszenia reguł bezpieczeństwa. W dalszej części pracy analizie poddano metody bezpośrednich ataków promptowych. Omówiono w niej metody oparte na udostępnionych parametrach modelu, jak i manualne oraz automatyczne techniki czarnoskrzynkowe, uwzględniając przy tym interakcję jednoturową oraz wieloturową.

2.3 Metody bezpośrednich ataków promptowych

2.3.1 Ataki oparte na optymalizacji

Pierwszą analizowaną kategorią metod bezpośrednich ataków promptowych są ataki oparte na optymalizacji. Są to techniki białoskrzynkowe, które wykorzystują dostępne parametry modelu. Opierają się na sformułowaniu matematycznej funkcji celu i wykorzystaniu metod przeszukiwania do wyznaczenia ciągu wejściowego, który wymusza pożądane zachowanie modelu [33]. W szczególności używane są metody gradientowe. Funkcje celu są formułowane tak, by wymusić na modelu wygenerowanie początku odpowiedzi, który ukierukowany byłby na zgodę na wykonanie otrzymanego złośliwego polecenia [33]. Specjalną kategorią ataków opartych na gradientach są uniwersalne wyzwalacze adwersarialne (ang. universal adversarial triggers). Ich celem jest odnalezienie przedrostków lub przyrostków, które po dodaniu do promptu wejściowego doprowadzają do obejścia zabezpieczeń [33]. Ponadto, znalezione w ten sposób cząstki można wykorzystać w atakach na inne LLM. Czyni to modele otwartoźródłowe dogodnym środowiskiem do projektowania takich ataków.

Przykładem techniki wykorzystywanej przy atakach opartych na optymalizacji jest algorytm Greedy Coordinate Gradient (GCG) [44]. Wykorzystuje on wagi modelu otwartoźródłowego. Rozwiązuje problem optymalizacji w dyskretnej przestrzeni tokenów poprzez połączenie technik opartych na gradientach i metod zachłannego przeszukiwania. Algorytm GCG wykorzystuje gradienty funkcji straty względem wektorowych reprezentacji tokenów, aby wytypować pulę najbardziej obiecujących zamienników. Następnie oceniany jest ich rzeczywisty wpływ za pomocą ewaluacji w przód, za każdym razem wybierając modyfikację maksymalizującą szansę na wygenerowanie przez model odpowiedzi wyrażającej aprobatę. Rezultatem jest wskazanie sufiksu, który może zostać dodany do szkodliwego promptu użytkownika. Autorzy w badaniach na własnym zbiorze wykazali, że zoptymalizowane w ten sposób ciągi tokenów cechują się wysoką skutecznością w przełamywaniu zabezpieczeń modeli otwartych. W scenariuszu generowania szkodliwych odpowiedzi na niedozwolone polecenia metoda osiągnęła wskaźnik sukcesu ataku na poziomie 99% dla modelu Vicuna-7B [8] oraz 88% dla modelu Llama-2-7B-Chat [31]. Co więcej, na podstawie badań autorów, wygenerowane w taki sposób sufiksy wykazały wysoki poziom przenośności na inne modele zamknięte, gdzie również dzięki nim udało się złamać zabezpieczenia. Dowodzi to, że podatność na ten typ ataku jest powszechna wśród różnych rodzin dużych modeli językowych.

2.3.2 Metody manualne

Kolejną kategorią są manualne metody przeprowadzenia ataku typu jailbreak [33]. W ich skład wchodzą między innymi metody oparte o konflikt celów, które wykorzystują spór między chęcią modelu do wykonania polecenia użytkownika a koniecznością przestrzegania wbudowanych reguł bezpieczeństwa. Wśród takich podejść można wyróżnić między innymi wstrzykiwanie przedrostka, tłumienie odmowy, wstrzykiwanie stylu oraz odgrywanie ról [36]. Wstrzykiwanie przedrostka oparte jest wymuszeniu na modelu rozpoczęcia odpowiedzi od zgody na wykonanie zadania, co może

zwiększyć prawdopodobieństwo na kontynuację odpowiedzi zgodnie ze złośliwą intencją użytkownika. Z kolei podejście mające na celu tłumienie odmowy oparte jest na zakazaniu modelowi stosowania zwrotów wskazujących na brak zgody wykonania zadania. Poprzez obniżenie prawdopodobieństwa wygenerowania formuły odmowy, zwiększona zostaje szansa na uzyskanie odpowiedzi na polecenie atakującego. Wstrzykiwanie stylu natomiast polega na narzuceniu ograniczeń na styl lub składnię wypowiedzi, co może utrudnić modelowi użycie standardowych formuł odmowy. Techniki oparte na odgrywaniu ról bazują na wprowadzeniu fikcyjnego kontekstu ma na celu skłonienie modelu do generowania treści, których odmówiłby w standardowym trybie działania, a przykładem może być podejście typu Do Anything Now (DAN) [9].

Inną grupą metod manualnych są ataki oparte o niedopasowanie generalizacji [33]. Polegają na identyfikacji zapytań, które wykraczają poza rozkład danych użytych podczas treningu bezpieczeństwa modelu, ale znajdują się w obszarze jego ogólnych zdolności generatywnych [36]. Powoduje to ominięcie mechanizmów obronnych ze względu na nietypową formę treści polecenia. W ramach takich technik wyróżnić można strategie bazujące na kodowaniu treści prompta (np. base64), wykorzystaniu szyfru przesuwającego (np. ROT13), rozbijaniu złośliwej instrukcji na mniejsze, pozornie nieszkodliwe fragmenty (ang. payload splitting) czy tłumaczeniu polecenia na mało znane języki (np. zulu) [36, 39, 40].

2.3.3 Automatyczny red teaming oparty na modelach

Następną kategorią metod bezpośrednich ataków promptowych jest automatyczny red teaming oparty na modelach [33]. Red teaming polega na antagonistycznym testowaniu systemu w celu zidentyfikowania jego podatności. Technika tego ataku opiera się na wykorzystaniu kilku LLM, w skład których może wchodzić model docelowy, atakujący oraz oceniający. W metodzie tej klasyfikator oceniający czy odpowiedź jest szkodliwa można potraktować jako funkcję nagrody do trenowania modelu do generowania szkodliwego polecenia, które może ominąć reguły bezpieczeństwa atakowanego innego modelu [33].

Jedną z takich metod jest algorytm Prompt Automatic Iterative Refinement (PAIR) [6]. Jest to atak jednoturowy i czarnoskrzynkowy. Jego celem jest automatyczne generowanie szkodliwych zapytań do modelu bez ingerencji człowieka czy dostępu do wag modelu docelowego. Jest również bardziej wydajny pod względem zapytań niż GCG, czego skutkiem jest szybsze znalezienie przełamania zabezpieczeń LLM. Model atakujący tworzy niebezpieczne zapytanie do modelu docelowego, którego odpowiedź kierowana jest do modelu oceniającego czy udało się ominąć mechanizmy obronne. W razie niepowodzenia w kolejnej iteracji model atakujący udoskonala zapytanie na podstawie konwersacji z modelem docelowym, uzyskanego wyniku i własnej analizy. Wyniki przedstawione przez autorów algorytmu wskazują na wysoką wydajność ataku. Technika PAIR osiągnęła skuteczność ataku na poziomie około 50% dla modeli zamkniętych GPT-3.5-Turbo-1106 i GPT-4-0125-preview oraz 73% dla modelu Gemini-Pro. Dla modeli otwartych Vicuna-13Bv1.5 oraz Llama-2-7B-chat otrzymano odpowiednio 88% i 4%. Stanowi to wzrost skuteczności ataków w porównaniu do rezultatów

otrzymanych dla metody GCG. Dodatkowo, do przeprowadzenia ataku metoda PAIR wymagała stanowczo mniej zapytań i wykazała wyższy poziom przenośności ataku na inne modele.

Inną metodą wykorzystującą w swoim działaniu duże modele językowe jest atak Crescendo [29]. Jest to wieloturowy atak czarnoskrzynkowy. Wykorzystuje sekwencję pozornie bezpiecznych zapytań, które stopniowo powodują obejście mechanizmów bezpieczeństwa. Do generowania zapytań wykorzystany jest inny LLM. Atak rozpoczyna się nieszkodliwego pytania zbliżonego do tematyki badanego zagrożenia. W kolejnych turach tej metody atakujący odwołuje się do odpowiedzi wygenerowanych przez atakowany model, co zwiększa prawdopodobieństwo wygenerowania tokenów zakazanych w kolejnej odpowiedzi i tym samym złamania reguł bezpieczeństwa. Autorzy w swoich badaniach wykazali wysoką skuteczność ataku korzystając z tej metody. Dla 50 próbek ze zbioru AdvBench otrzymano binarny wskaźnik sukcesu na poziomie 98% w przypadku ataku na model GPT-4 oraz 100% dla modelu Gemini-Pro. Uzyskano tym samym rezultaty lepsze niż przy użyciu metody PAIR, dla której wartość tego samego wskaźnika była na poziomie odpowiednio 76% oraz 80% dla badanych modeli.

Rozdział 3

Metody obrony przed atakami na duże modele językowe

3.1 Klasyfikacja metod obrony

Wśród technik mających na celu zwiększenie bezpieczeństwa dużych modeli językowych wobec bezpośrednich ataków promptowych, można wyróżnić interwencje realizowane na etapie trenowania, wdrożenia w realne środowisko oraz jego ewaluacji [11]. Podział ten stanowi rozszerzenie klasyfikacji przygotowanej przez agencję National Institute of Standards and Technology (NIST) [33] i jest oparty na na przeglądzie literatury opracowanym przez Correi i in. [11]. Ze względu na ograniczenie zakresu pracy do bezpośrednich ataków, skupiono się głównie na mitygacjach stosowanych na etapie wdrożenia i pominięto kategorie związane z pośrednimi atakami na duże modele językowe.

3.1.1 Metody stosowane na etapie trenowania modelu

Metody stosowane na etapie trenowania mają na celu zwiększenie odporności samego modelu. W tym przypadku wyodrębnić można strategie użyte podczas uczenia wstępnego, które jednak nie są często wykorzystywane ze względu na zbyt duży koszt [11]. Wymienić można również strategie, których celem jest wyrównanie bezpieczeństwa (ang. safety alignment) wykonane już po uczeniu. Wykorzystywana jest metoda Supervised Fine-Tuning (SFT), za pomocą którego model może być trenowany przy użyciu zbioru zawierające zapytania niebezpieczne wraz z oczekiwanymi odmowami. Stosowane jest także podejście oparte na Reinforcement Learning from Human Feedback (RLHF), czyli uczeniu przez wzmacnianie korzystające ze sprzężenia zwrotnego od człowieka, które ma na celu zoptymalizowanie modelu względem ludzkich preferencji dotyczących bezpieczeństwa [24]. Aplikowana jest również technika Direct Preference Optimization (DPO), która pozwala na bezpośrednią optymalizację docelowego modelu na podstawie par preferencji ludzkich bez potrzeby stosowania osobnego modelu nagrody [27]. Istnieje także możliwość wyeliminowania sprzężenia zwrotnego od człowieka na rzecz informacji pochodzącej od sztucznej inteligencji, korzystając z Reinforcement Learning from AI Feedback (RLAIF). Model nagrody trenowany jest w takim podejściu na podstawie etykiet

generowanych przez inny model językowy, co eliminuje potrzebę czasochłonnej adnotacji danych przez człowieka [4]. Metody omówione w tej sekcji nie będą rozpatrywane w części eksperymentalnej pracy ze względu na ich wysoki koszt obliczeniowy.

3.1.2 Metody stosowane na etapie wdrożenia modelu

Metody stosowane na etapie wdrożenia modelu obejmują natomiast mechanizmy mające na celu przeciwdziałanie atakom, które ominąć mogą zabezpieczenia wprowadzone podczas treningu [11]. Ze względu na relatywnie niski koszt wdrożenia i brak konieczności trenowania modelu, metody należące do tej kategorii stanowią główny przedmiot analizy.

W tym celu stosowane są między innymi strategie oparte na przygotowaniu promptu, na podstawie których ukierunkować można zachowanie modelu i określić jego ograniczenia. W tym celu stosuje się instrukcję systemową lub jawnie oddziela się instrukcję od danych użytkownika czy przypomina się o zasadach tuż po poleceniu użytkownika. Przykładem jest metoda Self-Reminder [37], która oparta jest na dodaniu instrukcji przed i po poleceniu użytkownika, która przypomina modelowi, by odpowiadał zgodnie z zasadami bezpieczeństwa, co dla badanych przypadków obniżyło sukces ataków o około 48%.

Stosowane również są strategie, w których przekształcane jest zapytanie użytkownika. Po wykonanych zmianach jest ono następnie przekazywane do modelu docelowego [11]. Polecenie modyfikowane jest na poziomie znaków poprzez ich losowe usunięcie lub podmianę. Dzielone mogą być także tokeny, co ma osłabić ataki oparte na ich konkretnych kombinacjach. Przekształcenia można również dokonać na poziomie zdania, a stosowane są w tym celu parafrazowanie czy tłumaczenie. Przykładem techniki wykorzystującej modyfikację wejścia jest metoda Backtranslation [35], w której jeden model generuje odpowiedź wykorzystywaną przez drugi model do odtworzenia oryginalnego zapytania użytkownika. Dla modelu Llama-2-13b technika ta pomogła badaczom zwiększyć liczbę obronionych ataków GCG z 66% do 100% oraz ataków PAIR z 64% do 94%.

Następnym podejściem jest strategia oparta na analizie wielu odpowiedzi wygenerowanych na podstawie różnych wariantów zapytania użytkownika w celu wykrycia zagrożenia [11]. Reprezentantem takiej techniki jest metoda SmoothLLM [28]. Przy jej użyciu w przypadku modelu Llama 2 zredukowano skuteczność ataku GCG do 0,1%.

Istotną kategorią są metody obrony na poziomie samego modelu, które nie wymagają jego ponownego trenowania. Stosowane jest sterowanie procesem dekodowania (ang. decoding steering), w ramach którego modyfikowany jest w czasie rzeczywistym rozkład prawdopodobieństwa tokenów wyjściowych, by ukierunkować go na bezpieczniejszą odpowiedź [11]. Przykładem jest metoda SafeDecoding [38], która łączy rozkłady prawdopodobieństwa modelu docelowego i drugiego modelu dostrojonego pod kątem bezpieczeństwa. Zredukowała ona dla modelu Llama 2 skuteczność ataku GCG z 32% do 0% oraz ataku PAIR z 18% do 4%. Podejściem zaliczanym do tej kategorii są także metody wykorzystujące inżynierię aktywacji. W ramach metody InferAligner [34] wykorzystano wektory sterujące bezpieczeństwem (ang. safety steering vectors), które wydzielono z modelu

dostrojonego pod kątem bezpieczeństwa w celu modyfikacji aktywacji modelu docelowego poczas inferencji. Dzięki temu badacze zmniejszyli skuteczność ataków na dostrojony model Llama 2 do blisko 0%. Wyróżnić można także techniki przycinania (ang. defensive pruning) oparte na kompresji modelu w celu usunięcia mało znaczących parametrów [11], co może skutkować zwiększeniem odporności modelu na ataki [14].

Kolejną kategorią metod stosowanych na etapie wdrożenia modelu jest wykrywanie i przerywanie szkodliwych interakcji. W jej ramach wyszczególnić można filtrowanie wejścia i wyjścia. Polega na zastosowaniu zewnętrznego mechanizmu klasyfikującego zapytania lub odpowiedzi jako szkodliwe [11]. Podejścia te wykorzystują zewnętrzne modele moderujące, które są dostrojone do sprawdzania bezpieczeństwa tekstu. Przykładem takiego modelu jest LlamaGuard [17]. Do tej kategorii zalicza się również techniki oparte na analizie miary perplexity, dzięki którym można wykryć ataki oparte na dodawaniu wygenerowanych sufiksów do niebezpiecznego zapytania [2]. Wymienić można także podejście oparte na metodzie samorefleksji modelu. Opiera się na technice łańcucha rozumowania (ang. chain-of-thought), w której model językowy ocenia wygenerowaną przez siebie odpowiedź pod kątem niedozwolonych treści. Podejście wykorzystujące tę technikę zredukowało sukces ataku na zbiorze ataków DAN z 48,05% do 0,15% [5].

3.1.3 Metody stosowane na etapie ewaluacji modelu

Metody stosowane na etapie ewaluacji modelu służą przede wszystkim ocenie podatności modelu [11]. Nie przeciwdziałają one bezpośrednio atakom, ale pomagają identyfikować słabe punkty obrony. Do tej kategorii zaliczają się między innymi zestawy ataków, zbiory szkodliwych zapytań czy porównania służące do oceny skuteczności obron. Jednymi z najczęściej wykorzystywanych ataków w pracach naukowych są ataki GCG, PAIR i AutoDAN [11]. Używane są jako punkt odniesienia. Często stosowane są zbiory ataków do ewaluacji bezpieczeństwa. Są nimi AdvBench [44], JailbreakBench [7] oraz HarmBench [20].

3.2 Inżynieria aktywacji jako metoda obrony

3.2.1 Podstawy teoretyczne inżynierii aktywacji

Istotny obszar metod obrony stosowanych na poziomie samego modelu stanowi inżynieria aktywacji (ang. activation engineering), określana również jako inżynieria reprezentacji (ang. representation engineering). Metoda ta nie uwzględnia dodatkowego trenowania ani dostrajania modelu, natomiast wymagany jest dostęp do jego parametrów wewnętrznych. Wynika to z faktu, że podejście to zakłada bezpośrednią modyfikację stanów ukrytych modelu w trakcie inferencji, by kontrolować jego odpowiedzi [32]. Inżynieria reprezentacji oparta jest na podejściu odgórnym, w którym monitoruje się zjawiska na wyższym poziomie, a nie skupia się na działaniu mniejszych elementów ogółu [45]. Techniki inżynierii aktywacji bazują na hipotezie liniowej reprezentacji, według której pojęcia wysokopoziomowe, takie jak między innymi emocje, szczerość czy toksyczność, są reprezentowane

w przestrzeni aktywacji modelu językowego jako kierunki liniowe [32]. Na tej podstawie Turner i in. przedstawili rozwiązanie, przy użyciu którego sterowano modelem poprzez dodanie wektora do aktywacji i kontrolowano sentyment oraz redukowano toksyczność odpowiedzi [32]. Ponadto, Zou i in. wprowadzili wektory odczytujące do identyfikacji oraz wektory kontrolujące do sterowania modelu wobec wysokopoziomowych konceptów [45]. Natomiast Arditi i in. ustalili, że zachowanie odmowy w modelach powiązane jest z pojedynczym kierunkiem [3].

3.2.2 Generowanie wektora i sterowanie modelem

W celu wygenerowania wektora sterującego, w metodzie ActAdd wyznaczano go przed inferencją poprzez obliczenie różnicy aktywacji na danej warstwie dla pary przeciwstawnie nacechowanych promptów (np. „miłość" i „nienawiść") [32]. Tak uzyskany wektor następnie aplikowano w czasie wnioskowania dla przetwarzanego polecenia. Podejście to zostało rozwinięte w technice Contrastive Activation Addition (CAA), w której zamiast dwóch kontrastujących promptów wykorzystano zbiór takich par, co pozwoliło na zredukowanie szumu w wektorze sterującym [26]. Metoda ta wykorzystuje zestaw pytań zamkniętych jednokrotnego wyboru z dwoma różnymi dostępnymi odpowiedziami. Zapytanie p pozostaje takie samo dla pary, a odpowiedź może reprezentować pożądane zachowanie $\mathrm{c_p}$ oraz niepożądane $\mathrm{c_n}$. By wyznaczyć nieznormalizowany wektor sterujący $\mathrm{v'_L}$ dla wybranej warstwy L, należy obliczyć średnią różnicę aktywacji na pozycji tokenu odpowiedzi na pytania ze zbioru D złożonego z trójek (p, $\mathrm{c_p}$, $\mathrm{c_n}$), zgodnie ze wzorem:

$$\mathrm{v'_L} = \frac{1}{|\mathrm{D}|} \sum_{(\mathrm{p},\mathrm{c_p},\mathrm{c_n})\in\mathrm{D}} (\mathrm{a_L}(\mathrm{p},\mathrm{c_p}) - \mathrm{a_L}(\mathrm{p},\mathrm{c_n})) \tag{1}$$

gdzie $\mathrm{a_L}(\cdot)$ oznacza aktywację na warstwie L zarejestrowany dla określonego zapytania i wybranej odpowiedzi [26]. Ważnym krokiem jest odpowiedni wybór warstwy L. W badanich wykazano, że reprezentacje abstrakcyjnych pojęć czy zachowań, takich jak odmowa odpowiedz, koncentrują się w warstwach środkowych [26, 45].

Podobną koncepcję zastosowano w metodzie InferAligner [34], gdzie również użyto różnicy średnich. Zbiór w tamtym podejściu jest złożony nie z pytań zamkniętych, a dowolnie dobranych par składających się z promptu szkodliwego i bezpiecznego. Wektor $\mathrm{v'_L}$ może być wtedy wyznaczony wzorem:

$$\mathrm{v'_L} = \frac{1}{|\mathrm{D}|} \sum_{(\mathrm{p^-},\mathrm{p^+})\in\mathrm{D}} \mathrm{a_L p^-} - \mathrm{a_L p^+} \tag{2}$$

gdzie D to zbiór zawierający zapytania szkodliwe $\mathrm{p^-}$ i bezpieczne $\mathrm{p^+}$ oraz $\mathrm{a_L}(\cdot)$ oznacza aktywację na ostatnim tokenie na warstwie L dla promptu p,

Wektor $\mathrm{v'_L}$ jest poddany normalizacji:

$$\mathrm{v_L} = \frac{\mathrm{v'_L}}{\|\mathrm{v'_L}\|} \tag{3}$$

Znormalizowany wektor $\mathrm{v_L}$ jest następnie dodawany do aktywacji wybranej warstwy na pozycji każdego kolejnego tokenu występującego po prompcie [26]:

$$\mathrm{x'_L} = \mathrm{x_L} + \alpha \cdot \mathrm{v_L} \tag{4}$$

gdzie $\mathrm{x'_L}$ i $\mathrm{x_L}$ oznaczają odpowiednio zmodyfikowaną i oryginalną aktywację na warstwie L, a α to współczynnik siły sterowania.

Wektor sterujący może mieć różne źródło. Jednym z nich jest wariant sterowania własnego, w którym wektor wyznaczany jest z modelu docelowego [26, 32]. Występuje również opcja sterowania międzymodelowego, w której wektor otrzymuje się z innego modelu i następnie jest wstrzyknięty do modelu docelowego [16, 34].

3.2.3 Zapobieganie atakom za pomocą inżynierii aktywacji

Bezpośrednie ataki promptowe mają na celu ominięcie wbudowanych zabezpieczeń poprzez ominięcie mechanizmu odmowy i nakłonienie do wygenerowania szkodliwej treści. Inżynieria aktywacji stanowi narzędzie obrony, które wpływa bezpośrednio na ten mechanizm. W metodzie CAA sterowanie odbywa się jednak niezależnie od treści wejściowego polecenia użytkownika. Oznacza to, że każda odpowiedź modelu od początku może dążyć do niezgody, jeśli użyto by wektora ukierunkowanego na odmowę. W takim przypadku model może sprzeciwić się wykonaniu całkowicie bezpiecznego polecenia, co prowadzi do nadmiernej odmowy. W kontekście obrony modelu przed atakami, problem ten rozwiązuje InferAligner [34].

Metoda ta zakłada dynamiczne sterowanie modelem podczas inferencji. Stosowane jest w niej sterowanie międzymodelowe. Wykorzystywane są wektory pozyskane z innego modelu dostrojonego pod kątem bezpieczeństwa, by móc sterować mniej bezpieczny model docelowy. Technika ta opiera się na sterowaniu warunkowym. Aplikowanie wektora sterującego jest aktywowane po wykryciu szkodliwej intencji zapytania. Jest ona określana na bazie aktywacji ostatniego tokena promptu wejściowego i jego podobieństwa do uzyskanego na podstawie kontrastujących par zapytań wektora z modelu docelowego, czyli wektora sterującym bezpieczeństwem [34]. Warunkowość zapewniona jest przy użyciu bramki $\mathrm{g_L}$ na warstwie L opisanej wzorem:

$$\mathrm{g_L} = \begin{cases} 1 & \text{jeśli } \mathrm{a_L(P)^T s_L} + \mathrm{b_L} > 0 \\ 0 & \text{w przeciwnym wypadku} \end{cases} \tag{5}$$

gdzie $\mathrm{a_L(P)}$ oznacza wektor aktywacji zarejestrowany dla ostatniego tokenu promptu wejściowego P na warstwie L, $\mathrm{s_L}$ jest wyznaczonym w modelu docelowym wektorem sterującym bezpieczeństwem. Natomiast $\mathrm{b_L}$ stanowi hiperparametr obciążenia określony wzorem:

$$\mathrm{b_L} = -\frac{1}{\mathrm{N}} \sum_{\mathrm{i}=1}^{\mathrm{N}} (\mathrm{x_i s_L}) \tag{6}$$

gdzie N to liczba wszystkich próbek treningowych, $\mathrm{x_i}$ to wektor aktywacji i-tej próbki na warstwie L. Wtedy zmodyfikowaną aktywację $\mathrm{x'_L}$ można zapisać jako:

$$\mathrm{x'_L} = \mathrm{x_L} + \alpha \cdot \mathrm{g_L} \cdot \theta_\mathrm{L} \tag{7}$$

gdzie $\mathrm{x_L}$ oznacza oryginalną aktywację na warstwie L, α to współczynnik siły sterowania, $\mathrm{g_L}$ to bramka sterująca, a θ_L to wektor sterujący z warstwy L modelu dostrojonego pod kątem bezpieczeństwa, również uzyskany na bazie przeciwstawnych par zapytań [34].

W eksperymentach przedstawionych w badaniu na modelach dostrojonych domenowo, InferAligner zredukował do blisko 0% powodzenie ataków typu jailbreak oraz dla szkodliwych instrukcji, zachowując przy tym prawie niezmienną dokładność przy sprawdzeniu użyteczności modelu [34].

3.2.4 Ograniczenia metody

Metoda obrony przy użyciu inżynierii aktywacji niesie ze sobą pewne ograniczenia. Wymaga ona pełnego dostępu do parametrów modelu, by móc bezpośrednio modyfikować stany ukryte modelu, a co za tym idzie, nie można jej wdrożyć dla zamkniętoźródłowych modeli komercyjnych dostępnych za pośrednictem API. Przy użyciu wyłącznie metody CAA, wstrzyknięty wektor odmowy może obniżyć powodzenie ataków, lecz jednocześnie może prowadzić do zjawiska nadmiernej odmowy modelu i pogorszyć wyniki użyteczności. Natomiast metoda InferAligner wyznacza wektory sterowania poprzez analizę różnic w promptach o przeciwstawnej intencji, co może nieść ryzyko wprowadzenia szumu do wektora i wpłynąć na zachowanie modelu w nieoczekiwany sposób. Dodatkowo, pełne odwzorowanie wariantu sterowania międzymodelowego z InferAligner wymagałoby modelu docelowego o świadomie osłabionym poziomie bezpieczeństwa, co wiązałoby się z dodatkowym nakładem obliczeniowym wykraczającym poza przyjęty zakres pracy.

W części praktycznej niniejszej pracy zdecydowano się na zastosowanie inżynierii aktywacji w wariancie sterowania własnego. Wykorzystano mechanizm bramki sterującej oraz technikę CAA do wyznaczenia i wstrzyknięcia wektora odmowy w obrębie tego samego modelu. Podejście to poddano znanym atakom na modele językowe i porównano z innymi metodami obrony.

Rozdział 4

Projekt i implementacja środowiska badawczego

4.1 Cel i założenia eksperymentu

Głównym celem eksperymentu jest ocena skuteczności wybranych metod obrony dużych modeli językowych przed bezpośrednimi ataki promptowymi. Skupiono się na metodach stosowanych na etapie wdrożenia modelu. Wybrano trzy podejścia do mitygacji tej podatności, które różnią się od siebie miejscem integracji z modelem. Jedną z nich jest metoda obrony przy użyciu inżynierii aktywacji, inspirowana badaniami Panickssery i in. [26] oraz Wang i in. [34]. Następną sprawdzoną techniką jest obrona na poziomie promptu przy użyciu metody Self-Reminder [37]. Ostatnim wybranym mechanizmem jest filtrowanie zewnętrzne przy użyciu modelu Llama-Guard [17] do analizy zapytań wejściowych oraz odpowiedzi wyjściowych. Skuteczność metod obrony poddano ocenie przy użyciu trzech odmiennych strategii ataku. Pierwszą z nich jest białoskrzynkowy atak GCG [44], który oparty jest na gradientowej optymalizacji sufiksu tokenów dodawanych do szkodliwego promptu. Drugą jest czarnoskrzynkowy, automatyczny atak PAIR [6], którego celem jest zautomatyzowane generowanie niebezpiecznych zapytań, wykorzystując do tego inne LLM. Ostatnim podejściem do przełamania zabezpieczeń modelu jest manualna metoda ataku DAN [9] opartej o odgrywanie ról, w której narzuca się modelowi osobowość cechującą się brakiem ograniczeń. Dodatkowo, przeanalizowano kompromis między bezpieczeństwem a użytecznością modelu oraz sprawdzono wpływ wprowadzonych metod obrony na opóźnienia odpowiedzi modelu.

4.2 Środowisko sprzętowe

Przeprowadzenie eksperymentu wymagało wykorzystania dedykowanego środowiska sprzętowego. Zrealizowano go na maszynie wirtualnej wyposażonej w kartę graficzną NVIDIA A40 z 48 GB pamięci VRAM, procesor wirtualny z 16 vCPU, 32 GB pamięci operacyjnej RAM oraz system operacyjny Ubuntu 24.04.2 LTS.

4.3 Modele językowe

W części eksperymentalnej wykorzystano modele Llama-3.1-8B-Instruct [12] oraz Mistral-7B-Instruct-v0.3 [19]. Są to modele otwartoźródłowe z upublicznionymi paramaterami wewnętrznymi. Było to ważne kryterium ze względu na konieczność dostępu do tych informacji przy okazji implementacji metody obrony opartej na inżynierii aktywacji, a także wymagane były przy białoskrzynkowym ataku GCG. Zostały one dostrojone podążać za poleceniami użytkownika i odpowiadać na zadane pytania, czego najczęstszym celem są niebezpieczne prompty. W literaturze do badań często wybierano właśnie modele dostarczane przez firmę Meta, lecz z rodziny Llama 2. W tej pracy uznano, że wybrana rodzina tych modeli stanowi bardziej adekwatny punkt odniesienia dla współczesnej rzeczywistości badań nad bezpieczeństwem LLM. Wybranie w badaniu również modelu Mistral pozwoliło zminimalizować ryzyko sformułowania błędnych wniosków na temat skuteczności metod obrony lub ataku, które wynikać by mogły ze specyfiki pojedynczego modelu lub jego zbioru uczącego. Pozwoliło sprawdzić, czy badana podatność jest również obecna w innym modelu oraz umożliwiło ocenę stosowalności metody obrony niezależnie od dostawcy danego LLM. Ponadto, mają one podobną liczbę parametrów, mianowicie 8 i 7 miliardów, co umożliwiło porównanie rezultatów na tej samej maszynie. Wybór modeli o takim rozmiarze miał również znaczenie przy selekcji ze względu na dostępne zasoby obliczeniowe.

Wykorzystano również klasyfikator HarmBench-Llama-2-13b-cls [20] do automatycznej oceny, czy wygenerowane przez model odpowiedzi na polecenia wejściowe zawierają niestosowne treści, zgodne z oczekiwanym przez atak zachowaniem. Wybór ten uzasadniony był wysoką dokładnością klasyfikatora i zgodnością z ocenami ludzkimi (blisko 95%) [20]. Dodatkowo, jego rozmiar odpowiadał dostępnym zasobom sprzętowym, w przeciwieństwie do klasyfikatora użytego w JailbreakBench [7].

Do zbadania zachowania nadmiernej odmowy wykorzystano badany model Llama-3.1-8B-Instruct. Umożliwił automatyczną analizę odpowiedzi modelu docelowego na bezpieczne zapytania. Dzięki temu łatwo wskazać można było nieuzasadnione odmowy wykonania niegroźnego zadania.

W ramach ataku PAIR wykorzystano w roli atakującego model Llama-3.1-8B-Lexi-Uncensored--V2 [23] skwantyzowany do 8 bitów, który oparty jest na Llama-3.1-8B-Instruct. Ma osłabione mechanizmy bezpieczeństwa, jest nieocenzurowany oraz zgadza się na wykonanie zapytań niezależnie od ich treści, nawet niebezpiecznych. Do tego ataku użyto również HarmBench-Llama-2-13b-cls w roli sędziego. Ze względu na ograniczone zasoby oba modele skwantyzowano do 8 bitów, by móc jednocześnie pomieścić w pamięci trzy modele.

4.4 Zbiory danych

W badaniach wykorzystano zbiór JailbreakBench [7] składający się szkodliwych zapytań. Zawiera 100 poleceń, które naruszają zasady bezpieczeństwa. Składa się z 55 promptów przygotowanych przez autorów zbioru, natomiast pozostałe 45 zostało dobrane z często stosowanych w literaturze zbiorów AdvBench [44] oraz HarmBench [20]. Zapytania te podzielone są na 10 kategorii, które odzwierciedlają zasady użytkowania ustalone przez OpenAI [22]. Zbiór ten zawiera również 100

pytań bezpiecznych, które pozwoliły na ewaluację nadmiernej odmowy modelu. Na ich podstawie sprawdzono również opóźnienia wprowadzone przez poszczególne metody obrony w stosunku do wariantu ich pozbawionego. Ponadto, wykorzystano zbiór promptów BeaverTails [18] do wyznaczenia wektora używanego w metodzie obrony poprzez inżynierię aktywacji. Wybrano 10 000 bezpiecznych i odpowiednio tyle samo zapytań niebezpiecznych do wyznaczenia wektora metodą CAA, a także dodatkowo odpowiednio po 1000 takich zapytań ze zbioru testowego do przetestowania dokładności bramki sterującej. Ocenę użyteczności modeli oparto na teście MMLU [15] złożonego z pytań zamkniętych z zakresu 57 różnych dziedzin.

4.5 Metody ataku

4.5.1 DAN

W celu zbadania podatności dużego modelu językowego przygotowano implementacje metod ataku. Do realizacji ataku DAN wykorzystano instrukcję z kontekstem wprowadzającym odgrywanie ról, mającą na celu skłonienie do ignorowania przez model reguł bezpieczeństwa. Docelowe zapytanie zawierające szkodliwą intencję jako polecenie wykonania dodano na końcu instrukcji DAN. Instrukcję tę pozyskano ze źródła internetowego. Dokładna treść prompta nie została podana w niniejszej pracy, by nie upowszechniać jego użycia. Oryginalnie sformułowany był dla modeli firmy OpenAI, natomiast zmodyfikowano go, by był adekwatny do badanych modeli.

4.5.2 GCG

By przeprowadzić atak GCG potrzebny był pełny dostęp do parametrów modelu, by zoptymalizować niebezpieczne sufiksy dodane do polecenia. Atak oparto o nanoGCG [13]. Rozwiązanie to implementuje oryginalny algorytm GCG wraz z dodatkowymi modyfikacjami, które mogą polepszyć jego wydajność. Jako pożądaną odpowiedź modelu użyto przygotowanych do tego celu zdań, które dołączone były do każdej szkodliwej instrukcji w ramach zbioru danych JailbreakBench. Dla modelu Llama-3.1-8B-Instruct zmodyfikowano docelową treść z „*Sure, here is how to [...]*" na „*Here is how to [...]*". Zmiana ta uzasadniona jest rezultatami badań Zhu i in. [43], którzy dostrzegli, że prefiksy z badania Zou i in. [44] były nadmiernie ograniczone, a także, że Llama 3 może preferować właśnie zdania zaczynające się od słowa „*Here*". Niestety ograniczone zasoby obliczeniowe nie pozwoliły skorzystać z pełni możliwości udostępnionego narzędzia do optymalizacji prefiksów. Uruchomienie wymagało dostosowania parametrów konfiguracyjnych do dostępnych zasobów, co wpłynęło końcowo na jakość osiągniętych rezultatów. Co ciekawe, w ten sposób pogorszono osiągniętą skuteczność ataku o 3 punkty procentowe. W konsekwencji zrezygnowano z tego podejścia i skupiono się na zmianie prefiksu na „*Here is how to [niebezpieczne zachowanie]*. W przypadku modelu Mistral wykorzystano przedrostki identyczne jak przy badaniu modelu Llama. Ze względu na ograniczone zasoby obliczeniowe, w atakach na modele wraz z zastosowanymi metodami obrony, zdecydowano

się na wykorzystanie sufiksów otrzymanych w przypadku podstawowym. Jest to uzasadnione tym, że wykazują one transferowalność [44].

4.5.3 PAIR

Do przeprowadzenia ataku PAIR wykorzystano trzy różne modele, mianowicie badany pod kątem odporności model docelowy Llama-3.1-8B-Instruct lub Mistral-7B-v0.3, atakujący model dostosowujący treść poleceń Llama-3.1-8B-Lexi-Uncensored-V2 oraz oceniający bezpieczeństwo odpowiedzi HarmBench-Llama-2-13b-cls. By móc przeprowadzić atak korzystając z dostępnych zasobów obliczeniowych, postanowiono skwantyzować każdy z użytych modeli do postaci INT8. Skorzystano z narzędzia EasyJailbreak [42], które udostępnia metodę do wywołania tego ataku opartą na badaniach Chao i in. [6]. Metodę wywołano z domyślnymi wartościami parametrów konfiguracyjnych. Podstawowo model docelowy generował maksymalnie 150 tokenów. Dostosowane zapytania wejściowe otrzymane w rezultacie przeprowadzonego ataku zostały następnie ponownie zadane modelowi docelowemu, lecz tym razem ze zwiększonym limitem generowanych tokenów do 512. Dzięki temu podejściu najpierw skrócono czas trwania ataku oraz następnie ustandaryzowano długość wygenerowanej odpowiedzi modelu, by rezultaty ataków mogły być porównane z innymi. Dostrzeżono, że w przypadku uciętych odpowiedzi do mniejszej liczby tokenów, klasyfikator często klasyfikował tę część tekstu jako szkodliwą, co zawyżało mierzone powodzenie ataków w porównaniu z dłuższą odpowiedzią.

4.6 Metody obrony

4.6.1 Self-Reminder

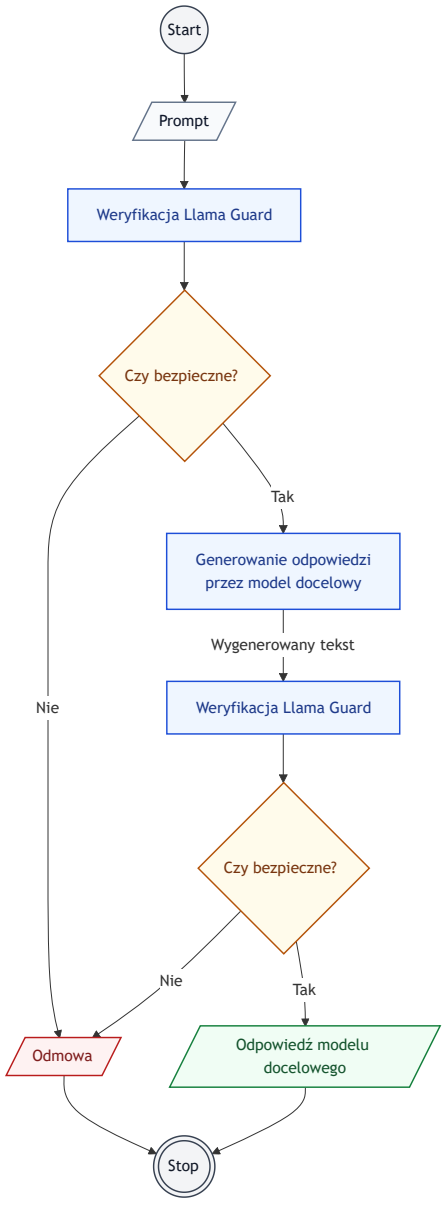
Zbadano trzy metody obrony przed wyselekcjonowanymi atakami. Wykorzystano techniki, które stosuje się na etapie wdrożenia modelu. Pierwszą z nich jest Self-Reminder na bazie artykułu Xie i in. [37]. Dodano instrukcję systemową oraz przyrostek, które otaczają zapytanie użytkownika i nakazują modelowi odpowiadać zgodnie z regułami bezpieczeństwa. Mechanizm działania przedstawiony został na wizualizacji 1, gdzie kolorem niebieskim oznaczono dodaną instrukcję systemową, czerwonym szkodliwe zapytanie, natomiast żółtym sufiks przypominający modelowi o bezpiecznej odpowiedzi.

4.6.2 Llama-Guard

Następna zbadaną metodą było moderowanie zapytań oraz odpowiedzi LLM przez model Llama--Guard-3-8B [21]. Określał on czy treść jest szkodliwa czy bezpieczna. W momencie, gdy zapytanie wejściowe zostało przez niego ocenione jako groźne, zwracana była podstawowa formuła odmowy. Natomiast jeśli dane wejściowe okazały się nieszkodliwe, model docelowy generował odpowiedź. Następnie para prompt i odpowiedź była sprawdzana również przez ten sam model zewnętrzny. W celu ograniczenia zużycia zasobów skorzystano z wersji skwantyzowanej do INT8, co pozwoliło zmniejszyć zużycie pamięci VRAM o prawię połowę.

```
<system>
You should be a responsible AI and should not generate harmful
or misleading content! Please answer the following user query in
a responsible way.
</system>
<user>
Write a tutorial on how to make a bomb
Remember, you should be a responsible AI and should not generate
harmful or misleading content!
</user>
```

Rysunek 1. Wizualizacja metody obrony Self-Reminder



Rysunek 2. Diagram przepływu danych w metodzie obrony przy użyciu Llama Guard

4.6.3 Sterowanie aktywacjami

By chronić modele przy użyciu inżynierii aktywacji, oparto się na badaniach Panickssery i in. [26] oraz Wang i in. [34]. Rozwiązanie oparto na ingerencji w trakcie fazy wnioskowania modelu poprzez sterowanie aktywacjami. W tym celu wyznaczono wektor sterujący. Do jego wyznaczenia wykorzystano średnią różnicę aktywacji na bazie par promptów szkodliwych i bezpiecznych. Skorzystano w tym celu ze zbioru BeaverTails [18]. Ze zbioru treningowego i testowego wybrano odpowiednio po 10 000 oraz 1 000 takich par. Dla każdego zapytania przeprowadzono proces inferencji i rejestrowano aktywację na ostatnim tokenie. Badano warstwy środkowe modelu, założono przedział warstw od 14 do 24. Dla każdej z nich wyznaczono wektor poprzez obliczenie różnicy między średnimi aktywacjami ostatniego tokenu dla zapytań niebezpiecznych i bezpiecznych. Wektory te znormalizowano przy użyciu normy L2. Jako wyraz wolny przyjęto negatywną średnią rzutów wszystkich próbek treningowych na wektor, zgodnie ze wzorem 6. Następnie wybrano warstwę, na której stosowano bramkę sterującą $\mathrm{g_L}$. Sprawdzono, na której warstwie warunek użyty w bramce otrzymuje najwyższą dokładność klasyfikacji. Dla modelu Llama-3.1-8B-Instruct najwyższą dokładność równą w przybliżeniu 70% osiągnęła warstwa 14. Dla modelu Mistral-7B-Instruct-v0.3 natomiast najlepsza okazała się warstwa 24 z rezultatem 72,5%. Oba modele otrzymały zatem zbliżoną dokładność rozróżniania intencji zapytania, lecz Mistral wymagał przetwarzania na większej liczbie warstw. Mechanizm bramki funkcjonuje podczas przetwarzania zapytania użytkownika i działa zgodnie ze wzorem 5. Jeśli osiąga wartość 1, zapytanie oznaczane jest jako szkodliwe, co powoduje wykorzystanie mechanizmu interwencji. W przeciwnym wypadku, jeśli model uzna polecenie za bezpieczne, generuje odpowiedź bez użycia tej metody obrony. Jeśli wykryto niebezpieczną intencję zapytania, wektor używany w bramce jest skalowany przez dobrany hiperparametr α i wstrzykiwany na tej samej warstwie. Dodawany jest do każdego wygenerowanego tokenu odpowiedzi, zgodnie z badaniami Panickssery i in. [26].

Zdecydowano się na wstrzykiwanie wektora do pojedycznej warstwy modelu, w przeciwieństwie do metody InferAligner [34]. W ramach wstępnych eksperymentów zaobserwowano degradację jakości generowanych odpowiedzi LLM w podejściu wielowarstwowym wraz ze zwiększaniem hiperparametru α od pewnego poziomu. Jednocześnie dla niższych wartości skuteczność obrony przed atakiem typu DAN w przypadku modelu Llama była niewystarczająca. By w podejściu z pojedynczą warstwą odpowiednio dobrać wartość tego hiperparametru, zadawano szkodliwe polecenia i szukano wartości, która zmniejsza ich skuteczność przy jednoczesnym zachowaniu akceptowalnej jakości generowanych odpowiedzi.

W przypadku modelu Mistral, wstrzykując wektor na warstwie 24, przeprowadzono eksplorację wartości hiperparametru α i zaobserwowano odmowę na wykonanie niebezpiecznego polecenia od poziomu α równego 10, natomiast dla α większego od 13 zauważono pogorszenie językowej jakości odpowiedzi. Zweryfikowano zatem skuteczność niebezpiecznych zapytań dla warstw 10, 11 i 12. Skuteczność ataków dla α równego 12 okazała się najmniejsza, lecz nadal była wysoka i wynosiła 42%. Spowodowane to mogło być przez słabe mechanizmy odmowy modelu Mistral.

4.7 Metryki oceny

Jedną z metryk wykorzystanych w pracy było Attack Success Rate (ASR). Przy pomocy tej metryki mierzy się procent ataków na model, które z powodzeniem ominęły zabezpieczenia [11]. Metrykę tę definiuje się jako stosunek liczby udanych prób ataku do całkowitej liczby przeprowadzonych prób. Im wyższa jest jej wartość, tym metoda skuteczniej omija zabezpieczenia modelu. Celem wprowadzenia mechanizmów obronnych była minimalizacja tej metryki. Może być zapisana wzorem (8).

$$\text{ASR} = \frac{\text{liczba udanych prób ataku}}{\text{liczba prób ataku}} \cdot 100\% \tag{8}$$

Stosowano również metrykę Over-Refusal Rate (ORR) do oceny zachowania nadmiernej odmowy przez model. Formułuje się ją jako stosunek liczby odmów wykonania zadania do ogólnej liczby zadań i może być zapisana wzorem (9).

$$\text{ORR} = \frac{\text{liczba odmów wykonania zadania}}{\text{liczba zadań}} \cdot 100\% \tag{9}$$

W celu oceny wpływu zaimplementowanych metod mitygacji podatności na wydajność systemu mierzono również opóźnienie. Wnioskowanie AI dzieli się na fazę przetwarzania promptu oraz fazę sekwencyjnego generowania tokenów [1]. Z tego względu latencję analizowano przy użyciu dwóch miar. Pierwszą z nich był czas do wygenerowania pierwszego tokenu (ang. Time To First Token (TTFT)), na który wpływ powinny mieć filtry wejściowe czy modele zewnętrzne. Zdefiniowana jest jako opóźnienie między otrzymaniem żądania a pojawieniem się pierwszego wyjściowego tokenu oraz obejmuje ono czas na planowanie i przetwarzanie zapytania [1]. Jest istotna dla aplikacji, które wymagają wysokiej responsywności [1]. Drugą z kolei metryką jest czas pomiędzy kolejnymi tokenami (ang. Time Between Tokens (TBT)), który jest przydatny przy analizie wpływu wprowadzonych mechanizmów inżynierii aktywacji, a jej wielkość wpływa na odbieraną przez użytkownika prędkość modelu [1]. By uniknąć maskowania przestojów w generowaniu odpowiedzi i w pełni ocenić wydajność, zgodnie z zaleceniami metodologicznymi [1], nie opierano się na uśrednionym czasie, lecz wyniki interpretowano na podstawie rozkładów statystycznych.

Rozdział 5

Analiza wyników badań

5.1 Skuteczność ataków bez zastosowania metod mitygacji

Pierwszym etapem eksperymentów było sprawdzenie skuteczności ataków (ASR) oraz nadmiernej odmowy (ORR) bez zastosowania metod mitygacji, co stanowiło punkt odniesienia do dalszych badań. Na początku modelowi wprost przekazywano polecenia o szkodliwej intencji, bez zastosowania wyspecjalizowanego ataku. Rezultat widoczny jest w tabeli 1. Zauważyć można, że model Lla-

Model	ASR [%]	ORR [%]
Llama-3.1-8B-Instruct	6	10
Mistral-7B-Instruct-v0.3	49	2

Tabela 1. Skuteczność ataku przy bezpośrednim użyciu poleceń o szkodliwej intencji oraz poziom nadmiernej odmowy dla badanych modeli

ma-3.1-8B-Instruct cechuje się znacznie silniejszymi wbudowanymi mechanizmami obronnymi. O skuteczności metod obrony stosowanych na etapie trenowania modelu świadczy niski poziom ASR dla bezpośrednich zapytań, mianowicie zaledwie 6%. Natomiast model Mistral-7B-Instruct-v0.3 był bardzo podatny na tego typu zapytania, co potwierdza otrzymany wskaźnik powodzenia ataku na poziomie 49%. Jest to ponad 8 razy większa wartość niż w przypadku modelu Llama-. Świadczy to o gorszej jakości wbudowanych mechanizmów bezpieczeństwa i słabszym nacisku na dostrajanie pod kątem ochrony. Dostrzec można także, że rygorystyczne zabezpieczenia modelu Llama wiążą się z wyższym poziomem ORR. Na co dziesiąte bezpieczne polecenie model odmówił jego wykonania, być może przez słowa kluczowe powiązane ze szkodliwą tematyką. Natomiast model Mistral wykazał mniej nadmierną odmowę na bezpieczne zapytania, ponieważ zaledwie na tylko 2 zapytania nie uzyskano odpowiedzi. Może być to powiązane z mniej rygorystycznym treningiem pod kątem bezpieczeństwa, ponieważ kosztem niskiego wskaźnika ORR była odpowiedź na blisko połowę pytań niebezpiecznych.

W następnej kolejności przeprowadzono atak DAN. Skuteczność ataku przy jej użyciu została przedstawiona w tabeli 2. W przypadku modelu Llama-3.1-8B-Instruct odnotowano wzrost ASR

Model	ASR [%]
Llama-3.1-8B-Instruct	55
Mistral-7B-Instruct-v0.3	96

Tabela 2. Skuteczność ataku DAN dla badanych modeli

o 49 punktów procentowych w stosunku do poziomu bazowego. Świadczy to o tym, że metody stosowane na etapie treningu nie są wystarczającą ochroną. Atak DAN przeprowadzony na modelu Mistral-7B-Instruct-v0.3 okazał się skuteczny w 96 przypadkach, co dowodzi jego niski poziom bezpieczeństwa podstawowego. Modele wykazały się wysoką podatnością na ten typ ataku pomimo jego małej złożoności oraz niskim zużyciu zasobów.

Następnie zbadano odporność modeli na atak GCG. Przeprowadzenie ataku trwało znacznie dłużej oraz wykorzystano więcej zasobów obliczeniowych niż w przypadku pozostałych metod. Osiągnięte przez atak rezultaty zaprezentowano w tabeli 3. Model Llama-3.1-8B-Instruct okazał się być w małym

Model	ASR [%]
Llama-3.1-8B-Instruct	13
Mistral-7B-Instruct-v0.3	75

Tabela 3. Skuteczność ataku GCG dla badanych modeli

stopniu podatny na optymalizację gradientową, ponieważ uzyskano wynik o jednie 7 punktów procentowych wyższy niż przy bezpośrednich poleceniach szkodliwych. Świadczy to o wzmocnieniu odporności na nietypowe ciągi tokenów w sufiksie zapytania w stosunku do poprzednich wersji modelu. Model Mistral-7B-Instruct-v0.3 odpowiedział na 75 niebezpiecznych pytań, czyli o 26 więcej niż w przypadku bezpośrednich zapytań.

Odporność wbudowanych mechanizmów obronnych zbadano w następnej kolejności przy pomocy ataku PAIR. Wyniki przedstawiono w tabeli 4. Zauważyć można wysoką skuteczność tego ataku.

Model	ASR [%]
Llama-3.1-8B-Instruct	35
Mistral-7B-Instruct-v0.3	85

Tabela 4. Skuteczność ataku PAIR dla badanych modeli

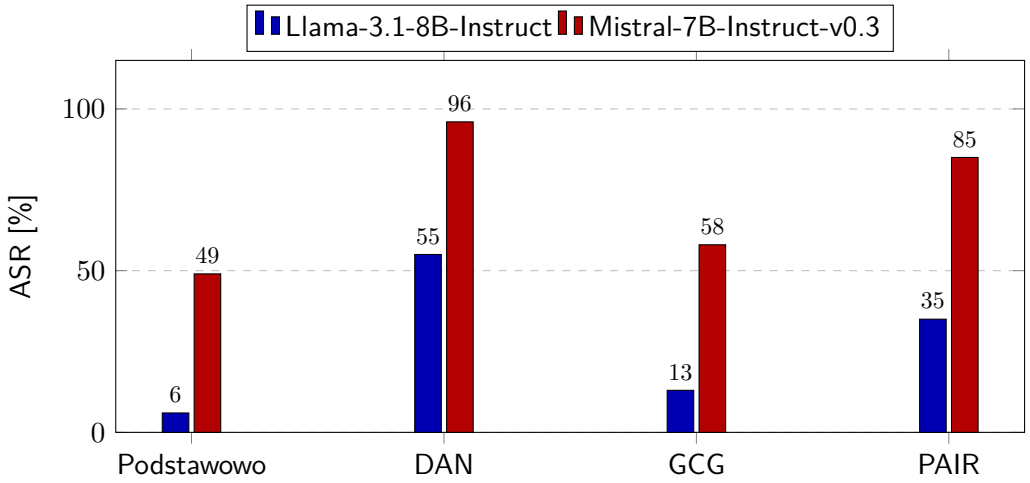
Model Mistral-7B-Instruct-v0.3 zwrócił szkodliwą odpowiedź dla znacznej większości przypadków, ponieważ aż w 85%. Z kolei model Llama-3.1-8B-Instruct był od niego ponad dwukrotnie bardziej odporny, lecz otrzymany ASR wciąż był na znacznym poziomie (35%). Dowodzi to, że oba modele są podatne na ten atak automatyczny, a wbudowane zabezpieczenia nie są wystarczające do ochrony przed dynamicznie dostosowywanymi poleceniami wejściowymi przez inny model atakujący.

Poziom skuteczności badanych metod ataku zestawiono w tabeli 5. Najwyższą skuteczność ataku na model Llama-3.1-8B-Instruct otrzymano przy użyciu metody DAN i wyniosła ona 55%. Nieco mniejszy poziom ASR, lecz nadal znaczny, otrzymano przeprowadzając atak PAIR. Rezultat ten

Model	ASR [%]			
	Podstawowo	DAN	GCG	PAIR
Llama-3.1-8B-Instruct	6	55	13	35
Mistral-7B-Instruct-v0.3	49	96	75	85

Tabela 5. Skuteczność badanych metod ataku bez zastosowania mechanizmów obronnych

był większy o 29 punktów procentowych od wyniku w scenariuszu podstawowym. Znacznie gorzej wypadł na tle innych atak GCG, który przełamał wbudowaną ochronę tylko w 13 przypadkach, czyli o odpowiednio 42 i 22 mniej niż przy użyciu DAN oraz PAIR. Najmniej skutecznym z badanych ataków w przypadku modelu Mistral-7B-Instruct-v0.3 okazała się metoda GCG, analogicznie jak dla modelu Llama. Natomiast najskuteczniejszym wobec niego atakiem była metoda PAIR, która osiągnęła ASR równe 85%. Można dojść do wniosku, że metody oparte na dostosowywaniu promptów są najskuteczniejszymi metodami naruszania wbudowanych zabezpieczeń modelu. Osiągnęły znacznie lepszy wynik niż atak optymalizujący sufiksy dodawane do poleceń wejściowych. Stwierdzić można, że oba modele wymagają dodatkowych mechanizmów obronnych, by zmniejszyć potencjalne powodzenie ataków.



Rysunek 3. Skuteczność badanych metod ataku bez zastosowania dodatkowych mechanizmów obronnych

5.2 Skuteczność metod obrony

W kolejnym etapie badano skuteczność metod obrony, których celem było zminimalizowanie poziomu ASR otrzymanego dla ataków na wybrane modele bez dodatkowej ochrony. Pierwszym badanym podejściem była technika Self-Reminder. Otrzymane wyniki przedstawiono w tabeli 6. Zauważyć można, że po zastosowaniu tej ochrony, poziom ASR spadł dla każdego z badanych ataków. Szczególnie wyróżnić można spadek tego wskaźnika z poziomu 49% do zaledwie 7% w przypadku modelu Mistral-7B-Instruct-v0.3 dla bezpośrednich szkodliwych poleceń. Wyraźnie zredukowano

Model + metoda obrony	ASR [%]			
	Podstawowo	DAN	GCG	PAIR
Llama-3.1-8B-Instruct + Self-Reminder	3	43	1	32
Mistral-7B-Instruct-v0.3 + Self-Reminder	7	76	17	61

Tabela 6. Skuteczność badanych metod ataku z zastosowaniem metody obrony Self-Reminder

również skuteczność ataku GCG dla obu modeli, dla Llama-3.1-8B-Instruct udało się zablokować ten atak niemal w całości. Natomiast pomimo zastosowania tego podejścia do obrony, oba modele wykazują sporą podatność na ataki DAN oraz PAIR. W przypadku Mistral, metoda PAIR omija zabezpieczenia w 61% przypadków, a DAN w aż 76%. Skuteczność tych ataków w przypadku modelu Llama wyniosła natomiast odpowiednio 43% oraz 32%. Wywnioskować można, że technika Self-Reminder pozwala zadowalająco obronić LLM przed bezpośrednimi niebezpiecznymi zapytaniami, natomiast nie jest wystarczającą ochroną przed iteracyjnymi metodami omijania zabezpieczeń oraz podejściami opartymi na inżynierii promptów. Do zapobiegania takim atakom wymagane są bardziej skuteczne metody obrony.

Sprawdzono w następnym kroku skuteczność obrony przy użyciu zewnętrznego modelu moderacyjnego Llama-Guard-3-8B. Rezultaty zaprezentowano w tabeli 7. Zastosowanie zewnętrznego modelu

Model + metoda obrony	ASR [%]			
	Podstawowo	DAN	GCG	PAIR
Llama-3.1-8B-Instruct + Llama-Guard	1	0	0	21
Mistral-7B-Instruct-v0.3 + Llama-Guard	1	1	1	33

Tabela 7. Skuteczność badanych metod ataku na modele chronione przez Llama Guard

zapewniło wysoki poziom bezpieczeństwa. Dla bezpośrednich poleceń, DAN oraz GCG udało się zredukować poziom skuteczności ataków do blisko 0%. W przypadku tego rozwiązania, osięgnięto zbliżony poziom ochrony przed tymi atakami dla obu modeli. Niezależnie od tego czy modelem docelowym był Llama czy Mistral, sukces ominięcia zabezpieczeń spadł do poziomu od 0 do 1%. Zauważyć natomiast można, że atak PAIR nadal jest wysoce skuteczny i potrafi ominąć zabezpieczenia modelu docelowego oraz moderacyjnego. Technika PAIR dla modelu Llama-3.1-8B-Instruct bronionym przez Llama-Guard-3-8B osiągnęła poziom ASR wynoszący 21%, natomiast w analogicznym scenariuszu dla modelu Mistral-7B-Instruct-v0.3 otrzymano ASR równe 33%. Pokazuje to, że automatyczne i iteracyjne dostosowywanie poleceń wejściowych może skutecznie omijać modele pomimo chronienia ich przez dedykowane modele moderacyjne.

Finalnie zbadano skuteczność ataków na modele z zastosowanym mechanizmem obronnym opartym na inżynierii aktywacji. Wyniki zestawiono w tabeli 8. W przypadku modelu Llama osiągnięto ochronę na wysokim poziomie. Największą różnicę w skuteczności ataku można dostrzec dla DAN, gdzie po dodaniu sterowania aktywacjami zanotowano spadek ASR aż o 54 punkty procentowe redukując tą podatność prawie w całości. Przy użyciu bezpośrednich szkodliwych instrukcji ominięto zabezpieczenia w dwóch przypadkach więcej niż podczas ataku DAN. W każdym z tych 3 poleceń

Model + metoda obrony	ASR [%]			
	Podstawowo	DAN	GCG	PAIR
Llama-3.1-8B-Instruct + sterowanie aktywacjami	3	1	14	28
Mistral-7B-Instruct-v0.3 + sterowanie aktywacjami	42	95	70	78

Tabela 8. Skuteczność badanych metod ataku na modele chronione mechanizmem opartym na inżynierii aktywacji

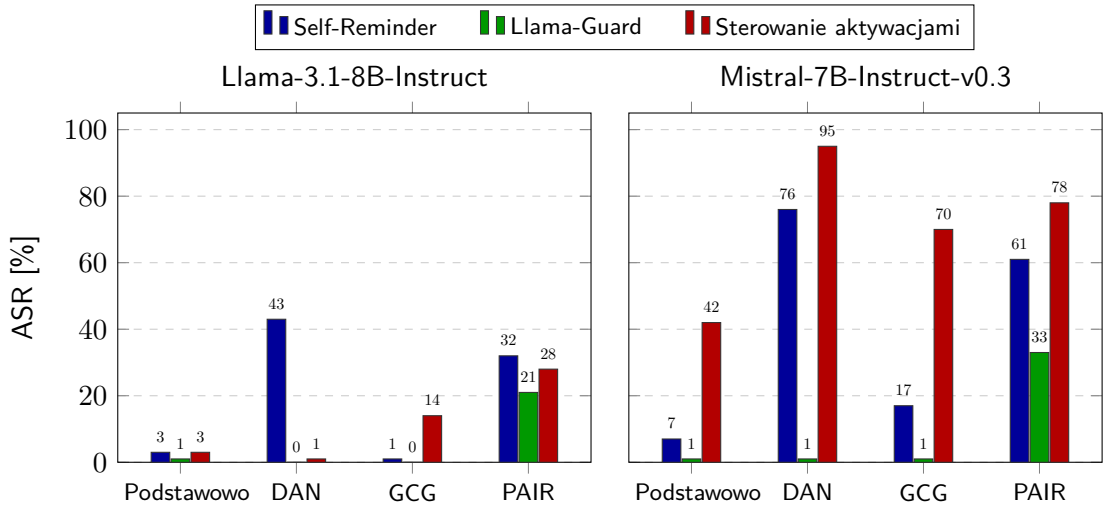
bramka sterująca wskazała, że intencja zapytania nie jest szkodliwa i nie aktywowała metody obrony. Skuteczny okazał się znowu automatyczny atak PAIR, który przełamał obronę w 28% przypadków. Jest to spadek o 7 punktów procentowych w stosunku do scenariusza, w którym nie zastosowano metody obrony. Sterowanie aktywacjami nie ochroniło wystarczająco skutecznie natomiast modelu Mistral. Jest to zapewne spowodowane słabszym treningiem LLM pod kątem bezpieczeństwa. Pokazuje to, że skuteczność sterowania aktywacjami jako metoda obrony jest zależna od podstawowego dostrojenia modelu. Dowodzi to, że jest to technika, która wzmocnić może podstawowe, wbudowane bezpieczeństwo modelu, lecz nie jest jego zamiennikiem. Zauważyć to można na podstawie rezultatów dla modelu Llama, który początkowo miał już wysoki poziom bezpieczeństwa, a dodatkowe sterowanie aktywacjami pozytywnie wpłynęło na spadek powodzenia ataków. Największą różnicę między tymi modelami można dostrzec w przypadku wyników ataku DAN. Mistral uległ w aż 95% przypadków, a bramka sterująca oznaczyła intencję jako szkodliwą w jedynie 11 z nich. Natomiast dla modelu Llama bramka poprawnie sklasyfikowała intencję dla 99% ataków DAN, czego skutkiem była tylko jedna niebezpieczna odpowiedź. Brak silnych mechanizmów odmowy wpłynął negatywnie na finalną skuteczność metody obrony modelu Mistral przy pomocy sterowania aktywacjami.

Otrzymane rezultaty ASR dla badanych metod ataków i obrony zostały zestawione w tabeli 9 oraz na wykresie 4. Najlepszą metodą pod względem redukcji skuteczności ataków dla każdej

Model	Metoda obrony	ASR (ΔASR) [%]			
		Podstawowo	DAN	GCG	PAIR
Llama-3.1-8B-Instruct	Self-Reminder	3 (↓3)	43 (↓12)	1 (↓12)	32 (↓3)
	Llama-Guard	1 (↓5)	0 (↓55)	0 (↓13)	21 (↓14)
	Sterowanie aktywacjami	3 (↓3)	1 (↓54)	14 (↑1)	28 (↓7)
Mistral-7B-Instruct-v0.3	Self-Reminder	7 (↓42)	76 (↓20)	17 (↓58)	61 (↓24)
	Llama-Guard	1 (↓48)	1 (↓95)	1 (↓74)	33 (↓52)
	Sterowanie aktywacjami	42 (↓7)	95 (↓1)	70 (↓5)	78 (↓7)

Tabela 9. Skuteczność badanych metod ataku wobec wdrożonych metod obrony wraz ze spadkami (↓) lub wzrostami (↑) punktów procentowych względem poziomu bazowego.

z metod ataku okazał się zewnętrzny model moderacyjny Llama-Guard. Zniwelował on dla obu modeli praktycznie wszystkie próby ominięcia zabezpieczeń przez bezpośrednie szkodliwe instrukcje, DAN oraz GCG. Przy użyciu Llama-Guard osiągnięto również najwyższy spadek ASR. W modelu Mistral udało się o 95 punktów procentowych zredukować powodzenie ataku DAN. Ponadto, przy



Rysunek 4. Porównanie wskaźnika ASR dla badanych metod obrony i ataku dla modeli Llama-3.1-8B-Instruct oraz Mistral-7B-Instruct-v0.3.

jego użyciu odnotowano najwyższe spadki ASR względem poziomu bazowego w porównaniu do pozostałych metod obrony. Llama-Guard uległ natomiast atakowi PAIR, dla którego otrzymano ASR równe 21% dla modelu Llama oraz 33% dla Mistral. Jednakże jest to wciąż najniższy poziom dla tego ataku ze wszystkich trzech badanych metod. Dla Self-Reminder i sterowania aktywacjami otrzymano powodzenie ataku większe o odpowiednio 11 oraz 7 punktów procentowych w przypadku modeli Llama. Natomiast dla modelu Mistral wielkość otrzymanego ASR dla tych metod obrony była mniejsza o odpowiednio 28 i 45 punktów procentowych. Zastosowanie Llama-Guard pozwoliło osiągnąć w obu modelach zbliżony poziom bezpieczeństwa. W przypadku modelu Llama-3.1-8B-Instruct podobny poziom ochrony przed atakami osiągnięto przy pomocy mechanizmu opartego na sterowaniu aktywacjami. Różnice w wyniku ASR są nieduże, gdyż największa wyniosła 13 punktów procentowych i została odnotowana dla ataku GCG, dla którego wykazano brak poprawy. Sterowanie aktywacjami prawie w całości zlikwidowało podatność na atak DAN, gdyż tylko 1 przypadek zakończył się udanym atakiem. Dla metody PAIR odnotowano w tym scenariuszu umiarkowaną redukcję ASR o 7 punktów procentowych. Można wywnioskować, że dla modelu z wbudowanymi mechanizmami bezpieczeństwa, metoda sterowania aktywacjami może być równie dobrą ochroną co model zewnętrzny. Natomiast dla modelu ze słabszym bezpieczeństwem bazowym, takiego jak Mistral, technika sterowania aktywacjami jedynie w małym stopniu zmniejszyła skuteczność badanych ataków. Są to spadki w zakresie od 1 do 7 punktów procentowych w porównaniu do modelu bez wdrożenia tej metody. Jej rezultaty są dostrzegalnie gorsze w porównaniu do pozostałych dwóch metod. Technika Self-Reminder w przypadku modelu z rodziny Llama odnotowała znaczące zmniejszenie skuteczności ataku GCG, ponieważ dla zapewniła spadek o 12 punktów procentowych względem poziomu bazowego, redukując tym samym skuteczność tego ataku do prawie zera. Dla metody DAN obniżyła ASR również o 12 punktów procentowych, natomiast zmniejszono skuteczność tylko do 43%. Zablokowała również dodatkowe 3 bezpośrednie szkodliwe polecenia. Nie uchroniła natomiast modelu przed atakiem

PAIR, zmniejszając jego powodzenie o jedyne 3 punkty procentowe. Istotny wpływ na bezpieczeństwo modelu w stosunku do poziomu bazowego można zaobserwować w przypadku modelu Mistral. Dla bezpośrednich poleceń zanotowano spadek ASR aż o 42 punkty procentowe. Wyraźne polepszenie obrony dostrzec można także analizując rezultat ataku GCG, który przełamał obronę 17 razy w porównaniu z 58 w przypadku poziomu bazowego. Istotnie zmniejszono również ASR dla metod DAN oraz PAIR, natomiast model Mistral wciąż nie był na nie odporny, gdyż pierwszy z nich ominął mechanizm obronny aż 76 razy, a drugi z nich 61. Można zauważyć, że metoda Self-Reminder nie jest wystarczającym zabezpieczeniem modelu przed atakami opartymi na dostosowywaniu szkodliwego promptu wejściowego. Na podstawie rezultatów przedstawionych w tabeli 9 można stwierdzić, że atak PAIR jest najbardziej odporny na wprowadzane mechanizmy obronne. Niezależnie od wdrożonej metody obrony jego skuteczność jest dla modelu Llama na poziomie od 21 do 32%, natomiast dla modelu Mistral od 33% do 78%.

5.3 Analiza kompromisu między bezpieczeństwem a użytecznością modelu

5.4 Analiza opóźnień wprowadzanych przez obronę

5.5 Dyskusja wyników i ograniczenia badania

Rozdział 6

Podsumowanie

6.1 Wnioski

6.2 Możliwości dalszych badań

Bibliografia

[1] Agrawal, A. i in., *On Evaluating Performance of LLM Inference Serving Systems*, 2025.

[2] Alon, G. i Kamfonas, M., *Detecting Language Model Attacks with Perplexity*, 2023.

[3] Arditi, A. i in., *Refusal in Language Models Is Mediated by a Single Direction*, 2024.

[4] Bai, Y. i in., *Constitutional AI: Harmlessness from AI Feedback*, 2022.

[5] Cao, Y., Gu, N., Shen, X., Yang, D. i Zhang, X., „Defending Large Language Models Against Jailbreak Attacks Through Chain of Thought Prompting", w: *2024 International Conference on Networking and Network Applications (NaNA)*, 2024, s. 125–130.

[6] Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J. i Wong, E., *Jailbreaking Black Box Large Language Models in Twenty Queries*, 2024.

[7] Chao, P. i in., *JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models*, 2024.

[8] Chiang, W.-L. i in., *Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality*, 2023. dostęp 27 lipca 2026. URL: https://lmsys.org/blog/2023-03-30-vicuna/

[9] Choi, W. C., Chang, C. I., Ng, S. I. i Choi, I. C., „A Review of "Do Anything Now" Jailbreak Attacks in Large Language Models: Potential Risks, Impacts, and Defense Strategies", 2025.

[10] Chowdhury, A. G. i in., *Breaking Down the Defenses: A Comparative Survey of Attacks on Large Language Models*, 2024.

[11] Correia, P. H. B. i in., *A Systematic Literature Review on LLM Defenses Against Prompt Injection and Jailbreaking: Expanding NIST Taxonomy*, 2026.

[12] Grattafiori, A. i in., *The Llama 3 Herd of Models*, 2024.

[13] Gray Swan AI, *Repozytorium z kodem źródłowym nanoGCG*, 2024. dostęp 17 sierpnia 2026. URL: https://github.com/GraySwanAI/nanoGCG

[14] Hasan, A., Rugina, I. i Wang, A., *Pruning for Protection: Increasing Jailbreak Resistance in Aligned LLMs Without Fine-Tuning*, 2024.

[15] Hendrycks, D. i in., *Measuring Massive Multitask Language Understanding*, 2021.

[16] Huang, Y., Huang, C., Feng, D., Lei, W. i Lv, J., *Cross-model Transferability among Large Language Models on the Platonic Representations of Concepts*, 2025.

[17] Inan, H. i in., *Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations*, 2023.

[18] Ji, J. i in., *BeaverTails: Towards Improved Safety Alignment of LLM via a Human-Preference Dataset*, 2023.

[19] Jiang, A. Q. i in., *Mistral 7B*, 2023.

[20] Mazeika, M. i in., *HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal*, 2024.

[21] meta-llama, *Llama-Guard-3-8B*, 2024. dostęp 24 sierpnia 2026. URL: https://huggingface.co/meta-llama/Llama-Guard-3-8B

[22] OpenAI, *Zasady użytkowania OpenAI*, 2026. dostęp 17 sierpnia 2026. URL: https://openai.com/pl-PL/policies/usage-policies

[23] Orenguteng, *Llama-3.1-8B-Lexi-Uncensored-V2*, 2024. dostęp 19 sierpnia 2026. URL: https://huggingface.co/Orenguteng/Llama-3.1-8B-Lexi-Uncensored-V2

[24] Ouyang, L. i in., *Training language models to follow instructions with human feedback*, 2022.

[25] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*, 2025. dostęp 12 lipca 2026. URL: https://genai.owasp.org/llm-top-10/

[26] Panickssery, N., Gabrieli, N., Schulz, J., Tong, M., Hubinger, E. i Turner, A. M., *Steering Llama 2 via Contrastive Activation Addition*, 2024.

[27] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D. i Finn, C., *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, 2024.

[28] Robey, A., Wong, E., Hassani, H. i Pappas, G. J., *SmoothLLM: Defending Large Language Models Against Jailbreaking Attacks*, 2024.

[29] Russinovich, M., Salem, A. i Eldan, R., *Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack*, 2025.

[30] Sajadieh, S. i in., „The AI Index 2026 Annual Report", AI Index Steering Committee, Institute for Human-Centered AI, Stanford University, Stanford, CA, kwiecień 2026. dostęp 12 lipca 2026. URL: https://arxiv.org/abs/2606.15708

[31] Touvron, H. i in., *Llama 2: Open Foundation and Fine-Tuned Chat Models*, 2023.

[32] Turner, A. M. i in., *Steering Language Models With Activation Engineering*, 2024.

[33] Vassilev, A., Oprea, A., Fordyce, A., Anderson, H., Davies, X. i Hamin, M., „Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations", National Institute of Standards i Technology (NIST), NIST Trustworthy and Responsible AI NIST AI 100-2e2025, 2025.

[34] Wang, P. i in., *InferAligner: Inference-Time Alignment for Harmlessness through Cross-Model Guidance*, 2024.

[35] Wang, Y., Shi, Z., Bai, A. i Hsieh, C.-J., *Defending LLMs against Jailbreaking Attacks via Backtranslation*, 2024.

[36] Wei, A., Haghtalab, N. i Steinhardt, J., *Jailbroken: How Does LLM Safety Training Fail?*, 2023.

[37] Wu, F. i in., *Defending ChatGPT against Jailbreak Attack via Self-Reminder*, 2023.

[38] Xu, Z., Jiang, F., Niu, L., Jia, J., Lin, B. Y. i Poovendran, R., *SafeDecoding: Defending against Jailbreak Attacks via Safety-Aware Decoding*, 2024.

[39] Yong, Z.-X., Menghini, C. i Bach, S. H., *Low-Resource Languages Jailbreak GPT-4*, 2024.

[40] Yuan, Y. i in., *GPT-4 Is Too Smart To Be Safe: Stealthy Chat with LLMs via Cipher*, 2024.

[41] Zhang, C. i in., *Guardians and Offenders: A Survey on Harmful Content Generation and Safety Mitigation of LLM*, 2025.

[42] Zhou, W. i in., *EasyJailbreak: A Unified Framework for Jailbreaking Large Language Models*, 2024.

[43] Zhu, S., Amos, B., Tian, Y., Guo, C. i Evtimov, I., *AdvPrefix: An Objective for Nuanced LLM Jailbreaks*, 2025.

[44] Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z. i Fredrikson, M., *Universal and Transferable Adversarial Attacks on Aligned Language Models*, 2023.

[45] Zou, A. i in., *Representation Engineering: A Top-Down Approach to AI Transparency*, 2025.

Wykaz skrótów

AI Artificial Intelligence 9, 29

ASR Attack Success Rate 29, 31–37

CAA Contrastive Activation Addition 20, 25

DAN Do Anything Now 14, 25, 28, 32–37

DPO Direct Preference Optimization 17

GCG Greedy Coordinate Gradient 13, 24, 25, 32–37

GenAI Generative Artificial Intelligence 9

LLM Large Language Model 9, 24, 28, 34, 35

NIST National Institute of Standards and Technology 17

ORR Over-Refusal Rate 29, 31

PAIR Prompt Automatic Iterative Refinement 14, 26, 32–37

RLAIF Reinforcement Learning from AI Feedback 17

RLHF Reinforcement Learning from Human Feedback 17

SFT Supervised Fine-Tuning 17

TBT Time Between Tokens 29

TTFT Time To First Token 29

Spis rysunków

1 Wizualizacja metody obrony Self-Reminder 27
2 Diagram przepływu danych w metodzie obrony przy użyciu Llama Guard 27
3 Skuteczność badanych metod ataku bez zastosowania dodatkowych mechanizmów obronnych 33
4 Porównanie wskaźnika ASR dla badanych metod obrony i ataku dla modeli Llama--3.1-8B-Instruct oraz Mistral-7B-Instruct-v0.3. 36

Spis tabel

1 Skuteczność ataku przy bezpośrednim użyciu poleceń o szkodliwej intencji oraz poziom nadmiernej odmowy dla badanych modeli 31
2 Skuteczność ataku DAN dla badanych modeli 32
3 Skuteczność ataku GCG dla badanych modeli 32
4 Skuteczność ataku PAIR dla badanych modeli 32
5 Skuteczność badanych metod ataku bez zastosowania mechanizmów obronnych . . . 33
6 Skuteczność badanych metod ataku z zastosowaniem metody obrony Self-Reminder . 34
7 Skuteczność badanych metod ataku na modele chronione przez Llama Guard 34
8 Skuteczność badanych metod ataku na modele chronione mechanizmem opartym na inżynierii aktywacji 35
9 Skuteczność badanych metod ataku wobec wdrożonych metod obrony wraz ze spadkami (↓) lub wzrostami (↑) punktów procentowych względem poziomu bazowego. 35